



Technical Assessment: React Native Developer

Project Title: Taskify Mobile Application (Task Management Mobile Application)

Timeframe: 72 Hours

Target Repository for API: [Taskify APIs GitHub Repo](#)



1. Objective

This assessment evaluates your frontend mobile application development proficiency, architectural decisions, code quality, UI/UX execution, and security practices.

You are required to build a feature-complete, secure **React Native** mobile application that integrates with a pre-built, self-contained RESTful backend API provided by Spearhead.

2. Architecture & Technical Stack Guidelines

To ensure flexibility while maintaining an objective evaluation baseline, you must adhere to the following stack specifications:

- **Mobile Frontend:** React Native (Expo or React Native CLI). **TypeScript is highly preferred.**
- **Backend API:** Node.js (Express) server application — [Clone/Fork the Repository Here](#).
- **Database:** Pre-configured SQLite database embedded directly within the provided backend application.

3. Core Functional Requirements

Part A: Provided RESTful API Reference

The backend service includes standard JWT (JSON Web Tokens) authentication controls and fully configured CORS handlers. Your app will need to consume the following endpoints:

Authentication Endpoints

- **POST /api/auth/register** — User signup with encrypted password storage.
- **POST /api/auth/login** — Authenticate user and return an access token.
- **POST /api/auth/social** — Process, match, and verify social login payloads.
- **POST /api/logout** — Invalidate the session on the backend.

Task Management (Protected by Auth Middleware)



- `POST /api/tasks` — Create a new task.
- `GET /api/tasks` — Fetch all tasks belonging to the authenticated user.
- `GET /api/tasks/:id` — Retrieve details for a single specific task.
- `PUT /api/tasks/:id` — Update task fields or toggle its completion status.
- `DELETE /api/tasks/:id` — Permanently remove a task.

Account & Profile Settings (Protected by Auth Middleware)

- `GET /api/user/profile` — Retrieve account profile details.
- `PUT /api/user/profile` — Modify profile data and securely update passwords.
- `DELETE /api/account` — Delete the authenticated user's account along with all associated historical data.

Part B: Mobile App Specifications

1. Authentication Layer

- **Standard Auth:** Build a clean email/password login and registration workflow complete with strict client-side form validation (e.g., regex checks, empty fields).
- **Social Auth:** Integrate at least **one** social login provider (Google, Apple, or GitHub authentication).
Note: If platform developer account restrictions delay setting up live production keys, clean sandbox configurations or a well-documented mock social handler within the UI is acceptable. Native integration remains preferred.
- **Token Security:** Store JWT access tokens safely using native keychain storage hooks (`expo-secure-store` or `react-native-keychain`). **Do not store raw JWTs in plain AsyncStorage.**

2. To-Do List Dashboard

- **Segmented Task List:** Implement a clean, scrollable view that separates tasks cleanly into **Pending** and **Completed** sections.



- **Dynamic Mutation:** Provide functional screens/modals to add and edit task parameters (Title, Description, and Target Completion Date).
- **Inline Actions:** Enable users to seamlessly toggle completion status or trigger quick deletions using mobile-centric inputs like horizontal swiping or long-press gestures.
- **State Management:** Demonstrate a robust global state structure across screens using the Context API, Redux Toolkit, or Zustand.

3. User Profile & Account Controls

- Display currently authenticated user metadata (Name, Email, Profile Picture URL).
- Provide forms to update personal profile details and edit the password safely.
- Implement a fully functional **Delete Account** trigger that cleanly consumes the backend purge route.
- Provide a **Log Out** mechanism that wipes all local secure tokens, clears global application state, and completely resets the navigation stack back to the login screen.

4. Bonus Features (Highly Regarded Stretch Goals)

Implement these elements to showcase exceptional full-stack mobile engineering capability:

- **Bonus 1: Smart Reminders & Push Notifications:** Allow users to assign specific dates/times to tasks. Implement native local push notifications ([expo-notifications](#) or [react-native-push-notification](#)) to alert the user when an assignment deadline is approaching.
- **Bonus 2: To-Do List Email Digest:** Integrate a frontend trigger that enables users to email themselves a formatted markdown/HTML summary of all outstanding pending tasks (you can utilize real transactional email wrappers or a sandboxed staging setup like Mailtrap).
- **Bonus 3: Offline Resilience:** Introduce a localized data caching mechanism. If the mobile app loses network connection, the user should still be able to browse their task historical frames, queue updates locally, and silently synchronize changes back to the Express server once connectivity is restored.



5. Evaluation Criteria

Your submission will be carefully graded based on the following engineering dimensions:

- **Code Architecture & Cleanliness:** Adherence to modular design principles. Clear separation of business logic, UI presentation components, network/API fetching layers, and state mutation blocks. Clean, descriptive variable naming conventions.
- **Security Configuration:** Secure handling of client-side local tokens, proper management of environment secrets, and clean error masking.
- **UI/UX Sophistication:** Device responsiveness across form factors, handling of safe areas/notches, smooth animation transitions, and handling of empty states, loading skeletons, and error fallbacks.
- **Error Resiliency:** User-friendly handling of network dropping, failed credentials, validation blocks, and general API breakdowns.

6. Submission Guidelines

Please package your codebase within **72 hours** of receiving this assessment brief. Your final submission must contain:

1. **Code Repository:** A link to your public GitHub repository containing your frontend code.
2. **Comprehensive documentation inside a `README.md` containing:**
 - Prerequisites and clear, step-by-step instructions to initialize and boot your React Native application environment.
 - An environment configuration block template (`.env.example`).
 - A brief engineering breakdown describing your chosen state management pattern and any architectural trade-offs made.
3. **Visual Verification:** A short screen recording (1–2 minutes long) walking through the operational app on an emulator, or a link to a compiled Android APK alongside a running web-build staging link.

